

Các thuật toán luồng mạng dựa trên tư tưởng phân tầng

Wang Xinshang

2007

Nguồn gốc: *Layered network-flow algorithms*

IOI China candidate team papers / 2007 / day 1 / Wang Xinshang.ppt

Tóm tắt

Bản trình chiếu giới thiệu ba thuật toán luồng cực đại dựa trên đồ thị phân tầng: MPLA, Dinic và MPM. Tư tưởng chung là dùng BFS trên đồ thị dư để lấy các mức, chỉ xét các cạnh đi từ mức i sang mức $i + 1$, rồi trong đồ thị phân tầng ấy tìm một luồng chặn. Sau mỗi pha, đường tăng luồng ngắn nhất trở nên dài hơn, nên số pha bị chặn bởi số đỉnh.

Từ khóa. Đồ thị phân tầng; đồ thị dư; luồng chặn; MPLA; Dinic; MPM.

1 Đồ thị dư

Cho mạng luồng $G = (V, E)$, nguồn s , đích t , sức chứa c và luồng hiện tại f . Với mỗi cạnh (u, v) , đồ thị dư có:

- cạnh thuận (u, v) có dung lượng dư $c(u, v) - f(u, v)$, nếu giá trị này dương;
- cạnh ngược (v, u) có dung lượng dư $f(u, v)$, nếu giá trị này dương.

Mỗi đường từ s tới t trong đồ thị dư là một đường tăng luồng. Lượng có thể tăng trên đường ấy là dung lượng dư nhỏ nhất của các cạnh trên đường.

2 Mức và đồ thị phân tầng

Trong đồ thị dư, mức của một đỉnh là khoảng cách ngắn nhất từ nguồn:

$$\text{level}(u) = \text{dist}_{G_f}(s, u).$$

Nguồn có mức 0. Đồ thị phân tầng chỉ giữ các cạnh dư (u, v) thỏa

$$\text{level}(v) = \text{level}(u) + 1.$$

Vì vậy mọi đường từ nguồn trong đồ thị phân tầng đều là một đường ngắn nhất trong đồ thị dư tới đỉnh cuối của nó.

Luồng chặn của một đồ thị phân tầng là một luồng làm cho trong đồ thị phân tầng không còn đường tăng luồng từ s tới t . Luồng chặn chưa chắc là luồng cực đại của toàn mạng; nó chỉ chặn hết các đường tăng luồng ngắn nhất của pha hiện tại.

3 MPLA

Thuật toán tăng theo đường ngắn nhất, được slide gọi là MPLA, lặp các bước:

1. khởi tạo hoặc cập nhật đồ thị dư;
2. chạy BFS để tính mức và đồ thị phân tầng; nếu không tới được đích, thuật toán kết thúc;
3. trong đồ thị phân tầng, liên tục dùng BFS tìm đường tăng luồng và tăng luồng cho tới khi không còn đường;
4. quay lại bước 2.

Một lần thực hiện bước 2 và 3 là một pha.

Sau khi tìm được luồng chặn trong một pha, mọi đường tăng luồng có độ dài bằng độ dài ngắn nhất cũ đã bị chặn. Các cạnh ngược mới sinh ra đi từ lớp sâu về lớp nông hơn, nên nếu còn đường tăng luồng thì đường ngắn nhất mới phải dài hơn. Vì độ dài đường đơn không vượt quá $n - 1$, số pha không vượt quá n .

Trong một pha của MPLA, mỗi lần tăng luồng làm bão hòa ít nhất một cạnh tầng, nên số lần tăng không vượt quá m . Nhưng mỗi lần lại tìm đường bằng BFS, vì thế tổng độ phức tạp thô là $\mathcal{O}(nm^2)$. Đây là lý do MPLA thường chỉ phù hợp với dữ liệu nhỏ hoặc dùng làm bước dẫn nhập.

4 Dinic

Dinic giữ cùng khung phân tầng, nhưng thay bước tìm đường lặp lại bằng DFS để tìm luồng chặn trong một pha. Các bước là:

1. BFS trên đồ thị dư để tính level;
2. nếu không tới được t , kết thúc;
3. DFS chỉ đi qua cạnh thỏa $\text{level}(v) = \text{level}(u) + 1$, đẩy luồng cho tới khi đồ thị tầng không còn đường từ s tới t ;
4. bắt đầu pha mới.

Khi DFS đi tới một đỉnh không còn cạnh hợp lệ ra ngoài, đỉnh ấy bị xem là cụt trong pha hiện tại và có thể bỏ qua về sau. Cài đặt thường dùng mảng “cạnh hiện tại” để mỗi cạnh tầng chỉ bị duyệt một số lần nhỏ. Khi tìm được một đường tới t , ta tăng luồng bằng giá trị nhỏ nhất trên đường và xóa hiệu quả những cạnh đã bão hòa khỏi pha.

Số pha của Dinic vẫn không vượt quá n . Trong một pha, chi phí tìm luồng chặn được chặn bởi $\mathcal{O}(mn)$ theo phân tích tổng quát trong slide: có nhiều nhất m lần bão hòa cạnh, mỗi đường dài không quá n , và các bước lùi do đỉnh cụt cũng bị chặn. Vì vậy độ phức tạp tổng quát là

$$\mathcal{O}(mn^2).$$

Trên mạng hai phía hoặc mạng dung lượng nguyên nhỏ, Dinic thường nhanh hơn đáng kể so với cận xấu nhất này.

5 Ứng dụng: lợi nhuận lớn nhất

Slide dùng bài NOI 2006 *profit* để minh họa mô hình cắt tối thiểu. Có N trạm, xây trạm i tốn P_i . Có M nhóm người dùng; nhóm j cần hai trạm A_j, B_j và đem lại lợi nhuận C_j . Cần chọn trạm và nhóm phục vụ để lợi nhuận ròng lớn nhất.

Dựng mạng:

- nguồn nối tới đỉnh nhóm X_j bằng cạnh sức chứa C_j ;
- đỉnh trạm Y_i nối tới đích bằng cạnh sức chứa P_i ;
- nếu nhóm j cần trạm A_j và B_j , nối $X_j \rightarrow Y_{A_j}$, $X_j \rightarrow Y_{B_j}$ với sức chứa INF.

Một lát cắt hữu hạn không thể cắt cạnh INF, nên nếu chọn lợi nhuận của một nhóm thì bắt buộc phải chọn cả hai trạm. Đáp án là

$$\sum_{j=1}^M C_j - \text{maxflow}.$$

Các số liệu trong bài gốc cho thấy MPLA dễ quá chậm trên test lớn, còn Dinic vượt qua nhanh và cài đặt ngắn hơn nhiều biến thể đẩy luồng trước.

6 Ứng dụng: trò chơi ma trận

Bài ví dụ thứ hai hỏi có tồn tại ma trận 0-1 với tổng hàng R_i , tổng cột C_j , và một số ô bị cấm. Mô hình luồng trực tiếp là:

- nguồn nối tới mỗi hàng X_i với sức chứa R_i ;
- mỗi cột Y_j nối tới đích với sức chứa C_j ;
- nếu ô (i, j) không bị cấm, nối $X_i \rightarrow Y_j$ với sức chứa 1.

Có lời giải khi và chỉ khi luồng cực đại bằng $\sum_i R_i = \sum_j C_j$. Mạng có thể có $\mathcal{O}(nm)$ cạnh, nên thuật toán luồng phải đủ mạnh. Slide nhấn mạnh rằng Dinic kết hợp một số kiểm tra tham lam đơn giản có thể xử lý tốt hơn nhiều so với thuật toán tăng đường ngắn nhất thô.

7 MPM

MPM cũng làm việc theo pha trên đồ thị phân tầng, nhưng chọn cách tìm luồng chặn khác Dinic. Với một đỉnh $u \neq s, t$, định nghĩa thông lượng:

$$\text{throughput}(u) = \min \left(\sum_{(v,u)} g(v,u), \sum_{(u,w)} g(u,w) \right).$$

Đây là lượng luồng tối đa có thể đi xuyên qua u nếu chỉ xét tổng dung lượng vào và ra trong đồ thị tầng.

Trong một pha, MPM chọn đỉnh có thông lượng nhỏ nhất, đẩy lượng luồng tương ứng từ đỉnh đó ngược về nguồn và xuôi tới đích, rồi xóa đỉnh hoặc cạnh đã nghẽn khỏi đồ thị tầng. Lặp quá trình này cho tới khi không còn đường từ s tới t , ta thu được luồng chặn. Cách cài đặt chi tiết phức tạp hơn Dinic, nhưng cùng dựa trên nhận xét rằng một pha chỉ cần xử lý đồ thị các đường ngắn nhất hiện tại.

8 Kết luận

Tư tưởng phân tầng giúp tách thuật toán luồng thành các pha rõ ràng. MPLA dễ hiểu nhưng chậm; Dinic là lựa chọn thực dụng trong thi lập trình vì ngắn, ổn định và nhanh trên nhiều mô hình; MPM cho một hướng khác để tìm luồng chặn nhưng thường khó cài hơn. Khi một bài toán có thể dựng thành cắt tối thiểu hoặc ghép hai phía có dung lượng, Dinic là công cụ nên nghĩ tới đầu tiên.